MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasile Brînză, FAF-242

Report

Laboratory Work No. 1
Operating Systems
Topic: The OS as a Resource Manager

Checked by:
Reiţman Patricia, *university assistant*

Chişinău 2026

Table of Contents

1 INTRODUCTION....3
1.1 Aim of the work....3
1.2 Working environment....3
2 PART 1. FILES AND DIRECTORIES....4
2.1 Navigation and inspection....4
2.2 Creating, copying, renaming and removing files....4
2.3 File ownership and permissions....4
2.4 Directories under /....5
2.5 Interesting output....5
3 PART 2. PROCESSES....6
3.1 Listing processes....6
3.2 Starting and stopping a background process....6
3.3 The /proc pseudo-filesystem....6
3.4 Process number 1....7
3.5 Interesting output....7
4 PART 3. MEMORY....8
4.1 Total and free RAM....8
4.2 Swap....8
4.3 Memory used by a process....8
4.4 Interesting output....8
5 PART 4. DEVICES AND STORAGE....9
5.1 Mounted filesystems....9
5.2 Block devices....9
5.3 Size of the laboratory folder....9
5.4 Devices as files....10
5.5 Mount information....10
5.6 “Everything is a file”....10
5.7 Interesting output....11
CONCLUSION....12
REFERENCES....13

1 INTRODUCTION

1.1 Aim of the work

The purpose of this laboratory work was to understand how an operating system manages the main resources of a computer. Using a Linux terminal on the provided virtual machine, a set of commands was executed in order to observe what happens with files, processes, memory and devices.

The main idea was not only to run commands, but to read their output and explain what the operating system is doing behind it. Each part of the work corresponds to one of the four core jobs of an operating system: file management (Part 1), process management (Part 2), memory management (Part 3) and I/O and storage management (Part 4). For every part, the answers to the “Observe” questions are given, together with one command output that was found interesting.

1.2 Working environment

All experiments were performed in a terminal on the provided Linux virtual machine; nothing had to be installed. The work was done under the user account `kynex`, inside the working folder `~/os-lab1`. According to the outputs collected during the work, the machine has 27 GiB of RAM and 2 GiB of swap, an NVMe disk whose root partition uses the `ext4` filesystem, and `systemd` as its first process.

Notes and outputs were collected in the file `report.md` while the commands were run; this document is the formal version of those observations.

2 PART 1. FILES AND DIRECTORIES

The operating system turns a raw disk into a tree of named files with owners and permissions. This part explores that tree and changes it.

2.1 Navigation and inspection

The work started by checking the current location and looking at the Linux filesystem, using the following commands:

```
pwd
ls -la /
ls -la ~
cd /etc && ls | head
```

The `/` directory is the top of the Linux filesystem. Many important directories are located directly under it, such as `/etc`, `/dev`, `/home` and `/usr`.

2.2 Creating, copying, renaming and removing files

Next, a directory for the laboratory was created and a small text file was used to test the basic file operations:

```
mkdir demo
cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
```

The file `note.txt` was created, a copy of it was made, the copy was renamed and finally removed. After each step the directory listing changed accordingly, which shows that the operating system keeps the file tree consistent with every operation.

2.3 File ownership and permissions

Ownership and permissions of the new file were inspected with `ls -l`:

```
ls -l note.txt
-rw-r--r-- 1 kynex kynex 24 Sep 28 16:45 note.txt
```

Figure 2.1: Output of ls -l note.txt

The output shows that the owner of the file is the user `kynex` and that the group is also `kynex`. Then the permissions were changed to owner-only access and back:

```
chmod 600 note.txt
ls -l note.txt
-rw-------
```

Figure 2.2: Permissions of note.txt after chmod 600

The ten characters at the start of the line are read as follows:

- The first character, `-`, means that `note.txt` is a normal file.
- The next three characters, `rw-`, are the permissions of the owner: the file can be read and written by

its owner.

- The next three characters, `---`, mean that members of the group have no permissions.
- The last three characters, `---`, mean that other users have no permissions either.

In simple terms, `chmod 600` makes the file accessible only to its owner, for reading and writing. Afterwards the permissions were restored with `chmod 644 note.txt`: the owner can still read and write the file, while the group and other users can only read it.

2.4 Directories under /

Two directories directly under the root directory were examined:

- `/etc` holds the configuration files used by Linux itself and by different programs and services.
- `/dev` holds special files that represent devices and device interfaces. It is one of the places where Linux treats system resources like files.

2.5 Interesting output

The output in Figure 2.1 was the most useful in this part, because a single command shows the permissions, the owner, the group, the size, the modification date and the name of the file at once.

3 PART 2. PROCESSES

Every running program is a process created, isolated and scheduled by the operating system. This part shows how to find processes, start one and stop it.

3.1 Listing processes

The currently running processes were listed and then counted:

```
ps aux | head
ps aux | wc -l
```

The count returned around 373 lines, so there were roughly 370 processes on the idle virtual machine. This number was surprising, because no hundreds of programs were started manually. It shows that most of the processes run in the background as part of the operating system and its services.

3.2 Starting and stopping a background process

A `sleep` process was started in the background, located, and then terminated:

```
sleep 300 &
jobs
ps -ef | grep sleep
kill <PID>
```

The shell printed the PID of the new process when it was started. The command `jobs` shows the background jobs from the point of view of the shell, while `ps -ef | grep sleep` finds the same process among all processes of the system. Finally, `kill <PID>` asks the operating system to terminate it. This demonstrates that every running process has its own PID, which is used to identify and manage it.

3.3 The /proc pseudo-filesystem

Another `sleep` process was started, and the information that the kernel exposes about it through `/proc` was displayed:

```
sleep 300 &
PID=$!
cat /proc/$PID/status | head -20
```

```
Name:   sleep
Umask:  0002
State:  S (sleeping)
Tgid:   8259
Ngid:   0
Pid:    8259
PPid:   5450
```

Figure 3.1: Part of /proc/<PID>/status for the sleep process

The PID of this process was `8259` and its parent process had PID `5450`. The most relevant line is `State: S (sleeping)`. It is expected, since the `sleep` command does not calculate anything: the process only waits, so the operating system does not give it CPU time until the waiting is over.

3.4 Process number 1

The first process has PID 1, and on the tested system it is managed by `systemd`. It is started during the boot

process and is responsible for starting and managing many system services. Before this laboratory work it was known that Linux runs many background processes, but not that there is one special first process, started very early, which helps to manage the rest of the system.

3.5 Interesting output

The output in Figure 3.1 was found the most interesting, because the `/proc` filesystem allows a running process to be inspected almost like a normal file.

4 PART 3. MEMORY

The operating system gives each process its own view of memory and shares the physical RAM between them. This part looks at the totals for the whole machine and at the memory of one process.

4.1 Total and free RAM

```
free -h
```

```
               total        used        free      shared  buff/cache   available
Mem:            27Gi       4.3Gi        20Gi       263Mi       2.6Gi        22Gi
Swap:          2.0Gi          0B       2.0Gi
```

Figure 4.1: Output of free -h

The virtual machine has 27 GiB of RAM in total. At the moment of the measurement about 4.3 GiB was used and about 20 GiB was free. The “available” value (22 GiB) is slightly higher than “free”, because Linux can reuse part of the memory currently occupied by caches when applications need it.

4.2 Swap

Swap is disk space that the operating system can use as additional virtual memory when it needs it. It is slower than normal RAM because it is stored on the disk, so having free RAM is generally preferable. The virtual machine has 2.0 GiB of swap configured, and 0 B of it was in use at the time of the measurement.

General memory information provided directly by the kernel was also checked with:

```
cat /proc/meminfo | head -6
```

4.3 Memory used by a process

The resident memory of a bare `sleep` process was checked as follows:

```
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
```

```
VmRSS:      2124 kB
```

Figure 4.2: Resident memory (VmRSS) of a sleep process

The `sleep` process uses about 2.1 MB of physical RAM. This was not surprising, because `sleep` is a very simple program that mostly just waits, so very little memory was expected.

4.4 Interesting output

The output in Figure 4.1 was chosen as the most useful, because it shows the total memory, the used memory, the free memory and the swap at the same time.

5 PART 4. DEVICES AND STORAGE

The operating system hides every disk, terminal and device behind uniform file-like interfaces. This is the “everything is a file” idea of UNIX, which is explored in this part.

5.1 Mounted filesystems

```
df -h
```

```
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           2.8G  2.1M  2.8G   1% /run
efivarfs        148K   94K   50K  66% /sys/firmware/efi/efivars
/dev/nvme0n1p5  192G   92G   91G  51% /
tmpfs            14G     0   14G   0% /dev/shm
tmpfs           5.0M   12K  5.0M   1% /run/lock
/dev/nvme0n1p1   96M   70M   27M  73% /boot/efi
tmpfs           2.8G  200K  2.8G   1% /run/user/1000
```

Figure 5.1: Output of df -h

The important line is the one for `/dev/nvme0n1p5`, which is mounted on `/`. It shows that the root filesystem is located on the device `/dev/nvme0n1p5`, has a size of 192 GB and is 51% used.

5.2 Block devices

```
lsblk
```

```
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0 953.9G  0 disk
├─nvme0n1p1 259:1    0   100M  0 part /boot/efi
├─nvme0n1p2 259:2    0    16M  0 part
├─nvme0n1p3 259:3    0 757.8G  0 part
├─nvme0n1p4 259:4    0   759M  0 part
└─nvme0n1p5 259:5    0 195.2G  0 part /
```

Figure 5.2: Output of lsblk

The main disk is `nvme0n1`, with a size of about 953.9 GB, divided into several partitions. The partition that contains `/` is `nvme0n1p5`, which matches the result of `df -h`.

5.3 Size of the laboratory folder

```
du -sh ~/os-lab1
```

```
12K     /home/kynex/os-lab1
```

Figure 5.3: Output of du -sh ~/os-lab1

The laboratory folder uses about 12 KB of disk space. On the first attempt the directory name was typed incorrectly (`~/osd-lab1`), and Linux returned a “No such file or directory” error; after correcting the command, the real `~/os-lab1` directory was measured.

5.4 Devices as files

```
ls -l /dev | head
```

```
total 0
drwxr-xr-x   2 root root              60 Sep 28 16:18 accel
crw-r--r--   1 root root       10,   235 Sep 28 16:18 autofs
drwxr-xr-x   2 root root             320 Sep 28 16:18 block
crw-rw----   1 root disk       10,   234 Sep 28 16:18 btrfs-control
drwxr-xr-x   3 root root              60 Sep 28 16:18 bus
crw-r--r--   1 root root            5260 Sep 28 16:18 char
crw--w----   1 root tty          5,     1 Sep 28 16:18 console
lrwxrwxrwx   1 root root              11 Sep 28 16:18 core -> /proc/kcore
```

Figure 5.4: Output of ls -l /dev | head

The entry chosen from this list is `/dev/console`. It represents the system console and provides a file-like interface for interacting with it.

On the first attempt the command `ls -l dev | head` was used, which was incorrect: `dev` without a leading `/` means a directory called `dev` inside the current directory. The correct path is `/dev`, and with it the command returned the expected device entries.

5.5 Mount information

```
mount | head
```

```
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
udev on /dev type devtmpfs
(rw,nosuid,relatime,size=14222752k,nr_inodes=3555688,mode=755,inode64)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,noexec,relatime,size=2853332k,mode=755,inode64)
efivarfs on /sys/firmware/efi/efivars type efivarfs (rw,nosuid,nodev,noexec,relatime)
```

Figure 5.5: Output of mount | head

Among the mounted filesystems, the following line is especially useful because it directly connects the physical storage device with the filesystem that is actually used:

```
/dev/nvme0n1p5 on / type ext4 (rw,relatime,errors=remount-ro)
```

Figure 5.6: Root filesystem entry from the mount output

It shows that the device `/dev/nvme0n1p5` is mounted as the root filesystem `/` and that it uses the `ext4` filesystem. The `/dev/console` entry from Section 5.4 stands for the system console.

5.6 “Everything is a file”

Based on what was observed, the phrase “everything is a file” means that Linux provides file-like interfaces for many different resources, including devices and system information. For example, `/dev/console` is not a normal text file, but Linux still represents it through a file-like interface. This allows programs to work with many resources using similar operations, such as reading and writing.

5.7 Interesting output

The output in Figure 5.6 was found the most interesting, because it links the physical storage device, the

mount point and the filesystem type in a single line.

CONCLUSION

This laboratory work helped to understand the operating system as a resource manager by actually observing four important resources in Linux: files, processes, memory and devices with storage. Instead of only reading about these resources, the real state of the system was inspected through the terminal, and each command output was interpreted.

Before this work, the operating system was seen mostly as the thing that lets a user open programs and files. Now it is clear that it is also responsible for organizing files, running processes, allocating memory and communicating with hardware, and that it does so constantly in the background. The most surprising observations were the large number of processes on an idle machine (roughly 370), the existence of the special first process managed by `systemd`, and the way `/proc` and `/dev` expose kernel information and devices as ordinary files.

In summary, the four resources managed by the operating system and one command that made each of them visible are given in Table 1.

Table 1: Resources managed by the OS and commands used to observe them

Resource	**Command**	**What it showed**
Files	`ls -l` / `chmod`	Owner, group and permissions of `note.txt`
Processes	`ps aux` / `/proc/<pid>/status`	About 370 running processes; state of a sleeping process
Memory	`free -h`	27 GiB of RAM, 2 GiB of unused swap
Devices and storage	`df -h` / `lsblk` / `mount`	Root filesystem on `/dev/nvme0n1p5` (ext4)

REFERENCES

1. Wikipedia, “Operating system,” Wikipedia.org. [Online]. Available: https://en.wikipedia.org/wiki/Operating_system

2. Wikipedia, “Everything is a file,” Wikipedia.org. [Online]. Available: https://en.wikipedia.org/wiki/Everything_is_a_file

3. Wikipedia, “procfs,” Wikipedia.org. [Online]. Available: https://en.wikipedia.org/wiki/Procfs

4. Wikipedia, “chmod,” Wikipedia.org. [Online]. Available: https://en.wikipedia.org/wiki/Chmod

5. Linux man-pages project, “proc(5) – process information pseudo-filesystem,” man7.org. [Online]. Available: https://man7.org/linux/man-pages/man5/proc.5.html